

JUSTIFICACIÓN TÉCNICA

Sistema de Gestión de Convocatorias Universidad Surcolombiana

Decisión de diseño

El backend se implementó como un monolito modular en Spring Boot, organizado por capas (controller, service, repository, model) en lugar de microservicios. Para el alcance de esta prueba, un monolito modular reduce la complejidad operativa (un solo despliegue, una sola base de datos, sin necesidad de orquestar comunicación entre servicios) sin sacrificar separación de responsabilidades. La capa de servicio se definió mediante interfaces con su implementación separada (UsuarioService / UsuarioServiceImpl, etc.), lo que permite sustituir la lógica de negocio sin modificar los controladores.

Modelo de datos y relación N:M

La relación entre Convocatorias y Categorías es de muchos a muchos. Se resolvió con una tabla intermedia (convocatoria_categoria) sin columnas propias adicionales, por lo que en JPA se mapeó con @ManyToMany y @JoinTable directamente sobre la entidad Convocatoria, evitando crear una tercera entidad innecesaria. Si en el futuro esa relación necesitara metadata propia (por ejemplo, fecha en que se asoció una categoría), se migraría a una entidad explícita sin afectar el resto del modelo.

Identificadores UUID

Todas las tablas usan UNIQUEIDENTIFIER (UUID) como clave primaria en lugar de enteros autoincrementales. Esta decisión facilita la futura migración a un esquema distribuido (microservicios o múltiples instancias de base de datos), ya que los identificadores se generan sin coordinación central y no colisionan al combinar datos de distintos orígenes.

Seguridad: JWT y autorización por rol

La autenticación se implementó con JSON Web Tokens, la autorización se definió a nivel de ruta y verbo HTTP (por ejemplo, GET /api/convocatorias abierto a cualquier autenticado, pero POST/PUT/DELETE restringido a ADMINISTRADOR), separando explícitamente quién puede leer de quién puede modificar cada recurso.

Validaciones de negocio en la capa de servicio

Las tres reglas de postulación (no duplicar, no postularse a convocatorias cerradas o no publicadas, no exceder el cupo) se validan en la capa de servicio antes de persistir, y se refuerzan con un constraint UNIQUE en base de datos como segunda barrera. Esto da mensajes de error claros y controlados al cliente, en lugar de exponer una excepción compleja.

Escalabilidad

Gracias a que la sesión del usuario viaja en el token JWT (y no se guarda en el servidor), el sistema puede crecer fácilmente agregando más servidores en paralelo cuando aumente la cantidad de usuarios, sin necesidad de cambiar el código.

Separación de lecturas y escrituras

El sistema distingue claramente entre las acciones que solo consultan información y las que la modifican. Esto lo hace más eficiente y permite que, si más adelante hay muchos usuarios consultando datos a la vez, esas consultas se puedan atender desde una copia adicional de la base de datos, sin afectar el funcionamiento general.

Camino hacia microservicios

El backend está dividido en módulos independientes por tema (usuarios, convocatorias, categorías, postulaciones, reportes), sin enredos entre ellos. Esto, junto con el tipo de identificador usado en cada registro, deja el sistema listo para que, si se necesita más adelante, pueda separarse en servicios independientes que se comuniquen entre sí.

Índices y consultas optimizadas

Se crearon índices sobre las columnas más consultadas (estado de convocatorias, estado de postulaciones, convocatoria_id en postulaciones) para mantener tiempos de respuesta estables a medida que crece el volumen de datos. Las consultas de reportes se resolvieron con JPQL agregando en base de datos (COUNT, GROUP BY) en lugar de traer todos los registros y agregarlos en memoria desde Java.

Frontend desacoplado

El frontend en Angular consume la API exclusivamente vía HTTP/REST, sin dependencias directas del backend más allá de la URL base configurada en las variables de entorno. Esto permite desplegar frontend y backend de forma independiente.

Mejoras futuras identificadas

- Agregar filtrado de datos por módulos, para que no se pueda consultar postulaciones ajenas aun conociendo el id del usuario.
- Guardar temporalmente los resultados de los reportes en una memoria rápida (como Redis), ya que cambian poco, pero se consultan seguido. Así el sistema respondería más rápido sin tener que recalcularlos cada vez.
- Migrar gradualmente a la opción de microservicios (ms-usuarios y ms-convocatorias) aprovechando que el modelo de datos con UUID ya lo permite sin reescritura.
- Agregar la opción de exportar información a Excel, permitiendo que el backend genere el archivo y el frontend ofrezca un botón para que el usuario descargue los datos consultados.